

Module 3 Project Rubric

TraceLens | Documentation & Spec Sprint | AIE 500 - 10x Coding with AI

C2.1-C2.3 | C2.1/C2.2 provisional until Module 4.

Document the repaired backend and specify both features before implementation; demonstrate useful project-specific AI instructions.

Evidence to prepare

- Write specs/evaluations.md and specs/alert-rules.md: goals, stories, acceptance criteria, data, endpoint shapes and errors/edge cases.
- Create docs/API_REFERENCE.md for implemented endpoints; document every function/class in models, API, storage and utilities.
- Create AGENTS.md or CLAUDE.md with TraceLens-specific standards and docs/agent-effect-note.md showing genuine before/after output.
- Keep setup, examples and seed instructions accurate. Commit both specs before feature code; build the features in Modules 4 and 5.

Provisional review

C2.1/C2.2 are provisional in Module 3 and finalized in Module 4. C2.3 is assessed here. Keep documentation synchronized thereafter.

Same competencies, independent project

Choose TraceLens **instead of PromptLab**. The videos demonstrate PromptLab; apply the methods independently. Both tracks require all 22 criteria to be Met, with no points, averaging or bonus credit.

No weekly capstone upload: keep one repository and authentic evidence for final review. Each module evidence group retains three attempts; returns do not consume one. See ASSESSMENT.md for revision, return and defense policy.

Version: TraceLens Criterion Set v1.1. Original project Modules 1-5 map to current Modules 2-6. The Syllabus governs policy; ALIGNMENT.md records clarifications.

Module 3 | Assessment criteria

C2 - Specification-Driven Development & Automated Quality Engineering

C2.1-C2.3 | C2.1/C2.2 provisional until Module 4.

Criterion	Where your evidence lives	What earns "Met"	Common mistakes
C2.1 - Your specs are specific enough Provisional until Module 4	specs/evaluations.md, specs/alert-rules.md	Each spec has overview and goals, user stories with acceptance criteria, data model changes, API endpoints with request and response shapes, and specified error conditions and edge cases. Every endpoint has at least one acceptance criterion you could write a test from directly.	- Requirements that name a feature without describing behaviour. - Specifying the happy path and leaving errors unstated. - Criteria you can't test, like "handles input correctly".
C2.2 - Your docs match your code Provisional until Module 4	README.md, docstrings in models.py / api.py / storage.py / utils.py, docs/API_REFERENCE.md	Docstrings on every function and class across all four source modules, with parameters, returns and exceptions matching the implementation. API reference documents every endpoint that exists - including PATCH and your Module 4 feature - with request examples, sample responses and error formats. README setup steps work on a clean clone.	- API reference describing an earlier draft of the spec. - Docs not updated after Module 4's feature work. - Setup steps missing an environment variable or dependency. - Docstrings that restate the function name.
C2.3 - Your agent instructions do something	AGENTS.md or CLAUDE.md; docs/agent-effect-note.md	The file states standards specific to TraceLens - its patterns, naming, error handling, testing expectations. The note shows one concrete before-and-after case where generated output changed because of it.	- Generic best-practice advice that would suit any project. - No evidence it affected any output. - Standards in the file that your own code contradicts.

Current tools and two features. Use AGENTS.md or CLAUDE.md with observed project-specific effects. Both Evaluation Scores and Alert Rules are required: specify both now, build one in Module 4 and the other in Module 5.

Source: TraceLens Criterion Set v1.1 (rubrics/CRITERIA.md), adapted from supplied v1.0 and aligned to the current PromptLab course package. All criteria are Met / Not Yet.